

9.1 File Inclusion

[This section corresponds to K&R Sec. 4.11.1]

A line of the form

```
#include <filename.h>
```

or

```
#include "filename.h"
```

causes the contents of the file `filename.h` to be read, parsed, and compiled at that point. (After `filename.h` is processed, compilation continues on the line following the `#include` line.) For example, suppose you got tired of retyping external function prototypes such as

```
extern int getline(char [], int);
```

at the top of each source file. You could instead place the prototype in a header file, perhaps `getline.h`, and then simply place

```
#include "getline.h"
```

at the top of each source file where you called `getline`. (You might not find it worthwhile to create an entire header file for a single function, but if you had a package of several related function, it might be very useful to place all of their declarations in one header file.) As we may have mentioned, that's exactly what the Standard header files such as `stdio.h` are--collections of declarations (including external function prototype declarations) having to do with various sets of Standard library functions. When you use `#include` to read in a header file, you automatically get the prototypes and other declarations it contains, and you *should* use header files, precisely so that you will get the prototypes and other declarations they contain.

The difference between the `<>` and `"` forms is where the preprocessor searches for `filename.h`. As a general rule, it searches for files enclosed in `<>` in central, standard directories, and it searches for files enclosed in `"` in the "current directory," or the directory containing the source file that's doing the including. Therefore, `"` is usually used for header files you've written, and `<>` is usually used for headers which are provided for you (which someone else has written).

The extension `".h"`, by the way, simply stands for "header," and reflects the fact that `#include` directives usually sit at the top (head) of your source files, and contain global declarations and definitions which you would otherwise put there. (That extension is not mandatory--you can theoretically name your own header files anything you wish--but `.h` is traditional, and recommended.)

As we've already begun to see, the reason for putting something in a header file, and then using `#include` to pull that header file into several different source files, is when the something (whatever it is) must be declared or defined consistently in all of the source files. If, instead of using a header file, you typed the something in to each of the source files directly, and the something ever changed, you'd have to edit all those source files, and if you missed one, your program could fail in subtle (or serious) ways due to the mismatched declarations (i.e. due to the incompatibility between the new declaration in one source file and the old one in a source file you forgot to change). Placing common declarations and definitions into header files means that if they ever change, they only have to be changed in one place, which is a much more workable system.

What should you put in header files?

- External declarations of global variables and functions. We said that a global variable must have exactly one *defining instance*, but that it can have *external declarations* in many places. We said that it was a grave error to issue an external declaration in one place saying that a variable or function has one type, when the defining instance in some other place actually defines it with another type. (If the two places are two source files, separately compiled, the compiler will probably not even catch the discrepancy.) If you put the external declarations in a header file, however, and include the header wherever it's needed, the declarations are virtually guaranteed to be consistent. It's a good idea to include the header in the source file where the defining instance appears, too, so that the compiler can check that the declaration and definition match. (That is, if you ever change the type, you do still have to change it in two places: in the source file where the defining instance occurs, and in the header file where the external declaration appears. But at least you don't have to change it in an arbitrary number of places, and, if you've set things up correctly, the compiler can catch any remaining mistakes.)
- Preprocessor macro definitions (which we'll meet in the next section).
- Structure definitions (which we haven't seen yet).
- Typedef declarations (which we haven't seen yet).

However, there are a few things *not* to put in header files:

- Defining instances of global variables. If you put these in a header file, and include the header file in more than one source file, the variable will end up multiply defined.
- Function bodies (which are also defining instances). You don't want to put these in headers for the same reason--it's likely that you'll end up with multiple copies of the function and hence ``multiply defined" errors. People sometimes put commonly-used functions in header files and then use `#include` to bring them (once) into each program where they use that function, or use `#include` to bring together the several source files making up a program, but both of these are poor ideas. It's much better to learn how to use your compiler or linker to combine together separately-compiled object files.

Since header files typically contain only external declarations, and should *not* contain function bodies, you have to understand just what does and doesn't happen when you `#include` a header file. The header file may provide the declarations for some functions, so that the compiler can generate correct code when you call them (and so that it can make sure that you're calling them correctly), but the header file does *not* give the compiler the functions themselves. The actual functions will be combined into your program at the end of compilation, by the part of the compiler called the *linker*. The linker may have to get the functions out of libraries, or you may have to tell the compiler/linker where to find them. In particular, if you are trying to use a third-party library containing some useful functions, the library will often come with a header file describing those functions. Using the library is therefore a two-step process: you must `#include` the header in the files where you call the library functions, *and* you must tell the linker to read in the functions from the library itself.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)